GLYVEX

Touchpad Gesture Recognition & Command Execution System

Final Semester Java Project

Technical Documentation · Technology Stack · Resources · Architecture · ML Plan

Project Concept

A Java desktop application that continuously monitors the laptop touchpad. Tapping all four corners in the configured sequence activates Gesture Mode. The user then draws a character or symbol on the touchpad. The trajectory is captured, normalized and classified using a machine-learning model trained on gesture/character data. The recognized symbol is mapped to a predefined computer action.

Team Members

Reg. No.	Name	Email
AP25110010712	Ashwath Raj Mulakalapalli	ashwath_m@srmap.edu.in
AP25110010120	BARATAM PRANEETH GUPTA	praneeth_baratam@srmap.edu.in
AP25110010905	Harsha Vardhan Kumar	harsha_jogi@srmap.edu.in
AP25110010841	Sai thanush reddy	thanush_bobbili@srmap.edu.in

Primary language: Java

Core domains: Touch Input · Machine Learning · Pattern Recognition · HCI · Desktop Automation

1. Executive Summary

GLYVEX is a touchpad-driven human–computer interaction system designed as a Java desktop application. Its central idea is to convert a deliberate physical interaction into a software command. The touchpad is monitored continuously. A four-corner tap sequence acts as the activation gate, after which a handwritten character or symbol can be drawn directly on the touchpad.

The captured drawing is represented as a time-ordered trajectory. The system preprocesses the trajectory to reduce variation caused by position, scale and sampling rate. A machine-learning classifier then predicts the gesture class. A controlled command registry maps that class to an approved desktop action.

The project combines hardware/input integration, event-driven Java programming, finite-state-machine design, data preprocessing, machine learning, local model inference, configuration, testing and controlled process execution.

> **Core pipeline:** Touchpad → Four-corner activation → Gesture capture → Preprocessing → ML recognition → Command mapping → Execution

The system is intentionally scoped around a small, reliable vocabulary of gestures rather than attempting unrestricted handwriting recognition.

2. Problem Statement & Motivation

Conventional desktop shortcuts generally rely on keyboard combinations, visible controls or predefined gestures. They are effective but have limited expressiveness and require the user to remember specific controls.

GLYVEX explores whether the touchpad can function as a compact command surface. A deliberate four-corner sequence signals that the user intends to issue a command; a subsequent drawn symbol specifies which command should be executed.

The engineering problem is not simply character recognition. The application must distinguish ordinary touchpad activity from the activation sequence, segment the drawing correctly, convert irregular sensor samples into a model-compatible representation, classify the result, reject uncertain predictions, and safely execute the mapped action.

> **Academic focus:** integration of HCI, event-driven programming, state machines, ML and operating-system interaction.

3. Objectives and Scope

Objective	Implementation target	Evidence
Touch monitoring	Acquire supported touchpad events	Raw input prototype
Activation	Detect all four corners in sequence	State transition
Gesture capture	Record x/y/time trajectory	Stored trajectory
Preprocessing	Normalize and resample	Fixed representation
ML recognition	Classify supported symbols	Class + confidence
Command mapping	Resolve predicted class	Registry lookup
Execution	Run approved local action	Successful launch
Configuration	Change mappings without code edits	Config/UI
Testing	Verify individual and integrated layers	Test suite

Scope limitation: The first prototype should support a small command vocabulary. Unrestricted shell execution, cloud inference and universal handwriting recognition are outside the core scope.

4. Functional Requirements

- FR-01 — The application shall remain active while the user is working on the desktop.
- FR-02 — The application shall receive supported touchpad events through the selected input mechanism.
- FR-03 — The system shall identify taps within four configurable corner regions.
- FR-04 — A valid four-corner sequence shall activate Gesture Mode.
- FR-05 — Gesture Mode shall capture the user's drawing trajectory.
- FR-06 — Capture shall terminate on release, timeout or cancellation.
- FR-07 — The trajectory shall be transformed into the model's required representation.
- FR-08 — The ML layer shall return a supported class and confidence where available.
- FR-09 — The predicted class shall be resolved through a command registry.
- FR-10 — Only explicitly allowed commands shall be executable.
- FR-11 — Low-confidence or unsupported gestures shall be rejected.
- FR-12 — The application shall provide state feedback and recover from errors.

5. Non-Functional Requirements

Area	Requirement
Performance	Local recognition should feel responsive for a small gesture vocabulary.
Reliability	Invalid input must return the system to a usable state.
Safety	Execution is restricted to an allowlist.
Privacy	Raw trajectories remain local by default.
Maintainability	Input, processing, ML, command and UI modules remain separated.
Portability	OS-specific input code is isolated behind an adapter.
Testability	Recorded trajectories can be replayed without hardware.
Usability	Activation and recognition states are understandable.
Extensibility	New classes and mappings can be added without redesigning the system.

These requirements turn the concept into measurable engineering criteria suitable for implementation and viva evaluation.

6. Overall System Architecture

Layer	Responsibility	Representative component
Input Adapter	Acquire touchpad events	OS/native bridge
Gesture State Machine	Activation and capture	IDLE / ARMING / CAPTURE
Trajectory Processor	Clean raw samples	Filtering / normalization
Feature Layer	Build model representation	Coordinates / deltas / geometry
ML Inference	Predict class	Java-compatible runtime
Command Router	Resolve class	Allowlisted registry
Executor	Perform action	Java process/file APIs
UI / Config	Status and mappings	JavaFX
Testing / Logging	Diagnostics	JUnit / SLF4J

Data flow: Touch Event → Activation → Gesture Capture → Preprocessing → Features → ML Prediction → Confidence Check → Command Lookup → Execution

7. Touchpad Input Acquisition

Raw touchpad access is the most important feasibility risk. Standard JavaFX or Swing APIs do not universally expose low-level touchpad coordinates and contact events. GLYVEX therefore treats hardware input as an integration boundary.

Approach	Advantage	Limitation	Role
Java UI events	Simple	May lack raw touch data	Use only if verified
Native OS bridge	Detailed input	Platform-specific	Preferred
Helper process	Separates native code	IPC complexity	Fallback
External touch device	Predictable data	Changes scope	Not preferred

The final implementation documentation should state the target operating system, input source, driver/interface and Java-to-native communication method. This is a genuine engineering constraint, not a detail that should be hidden.

> **Feasibility gate:** prove raw coordinate acquisition before spending significant time on the final UI or model.

8. Four-Corner Activation Mechanism

The touchpad is divided conceptually into four corner zones. Each zone has a configurable tolerance radius. The initial prototype can use a fixed order; configuration can be added later.

Current state	Event	Next state
IDLE	First valid corner	ARMING
ARMING	Correct next corner	ARMING
ARMING	All four corners complete	GESTURE_MODE
ARMING	Wrong corner / timeout	IDLE / restart
GESTURE_MODE	Drawing starts	CAPTURING
CAPTURING	Release / timeout	PROCESSING
PROCESSING	Accepted prediction	EXECUTING
PROCESSING	Low confidence	CANCELLED
EXECUTING	Action complete	IDLE

A timeout is necessary so that a partial sequence cannot remain active indefinitely.

9. Gesture State Machine

The application should use an explicit finite-state machine instead of unrelated boolean flags such as isActivated, isDrawing and isProcessing. This makes invalid transitions visible and testable.

State	Purpose	Typical data
IDLE	Normal monitoring	None
ARMING	Four-corner sequence	Corner index + timestamps
GESTURE_MODE	Ready for drawing	Activation timestamp
CAPTURING	Collect stroke	Touch sample list
PROCESSING	Preprocess and classify	Feature vector
EXECUTING	Run mapped command	Command identifier
CANCELLED	Discard gesture	Reason
ERROR	Recover safely	Error details

> Every state should have explicit entry, exit and recovery behavior.

10. Gesture Data Representation

The minimal raw representation is an ordered sequence of **(x, y, t)** samples, where x/y are coordinates and t is a timestamp.

Feature	Meaning	Potential use
x, y	Position	Spatial shape
Δx, Δy	Movement	Direction
Δt	Time difference	Speed/timing
Path length	Cumulative movement	Scale-independent descriptor
Direction	Local movement angle	Shape discrimination
Bounding box	Min/max coordinates	Normalization
Aspect ratio	Width / height	Geometric feature
Stroke count	Number of strokes	Optional class feature

Pressure, contact identifiers or other sensor fields should only be included if the selected input interface exposes them reliably.

11. Preprocessing Pipeline

- Normalize coordinates to a common range.
- Translate the gesture so absolute touchpad position has limited influence.
- Normalize scale while preserving important shape characteristics.
- Resample trajectories to a fixed number of points.
- Optionally smooth sensor noise without destroying corners.
- Calculate deltas, direction, path length and geometric features.
- Reject empty, extremely short or malformed trajectories.
- Use exactly the same preprocessing during training and inference.

The preprocessing pipeline is part of the model definition. A classifier trained on one representation and deployed with another representation can fail even when the underlying model is otherwise appropriate.

12. Machine-Learning Strategy

The ML task is multiclass classification: each supported symbol is one class.

Model family	Strength	Complexity	Role
k-NN	Very simple baseline	Low	Reference
Decision Tree	Interpretable	Low	Baseline
Random Forest	Robust tabular model	Medium	Candidate
SVM	Strong compact-space classifier	Medium	Candidate
Small MLP	Nonlinear decision boundaries	Medium	Optional
Sequence model	Learns temporal structure	High	Advanced extension

> Recommended workflow: establish a simple baseline → measure errors → refine features/data → select final model based on evidence.

13. Dataset Requirements

The dataset must represent the actual modality used by the deployed system. A public image dataset can be useful for concept development, but it is not automatically equivalent to touchpad trajectories.

Property	Recommended practice
Classes	Start with roughly 6–10 symbols.
Samples	Collect multiple examples from multiple users.
Variation	Different speeds, sizes, positions and natural styles.
Labels	One unambiguous class per trajectory.
Splits	Train / validation / test with no leakage.
User-independent test	Reserve some users for evaluation where possible.
Augmentation	Only transformations that preserve class identity.

The final report should document dataset source, collection procedure, number of classes, sample counts, preprocessing and split methodology.

14. Training and Evaluation

Every experiment should record its dataset version, preprocessing settings, model type and hyperparameters.

Metric	Purpose
Accuracy	Overall fraction of correct predictions
Precision	Reliability of positive predictions
Recall	Ability to identify actual class samples
F1-score	Combined precision/recall measure
Confusion matrix	Shows class-to-class confusion
Inference latency	Measures interactive response time
Rejection rate	Measures safe handling of uncertain gestures

User-independent testing is particularly useful because it tests whether the model recognizes the gesture concept rather than merely memorizing one user's drawing habits.

15. Java Technology Stack

Layer	Recommended technology	Purpose
Language	Java 17+ / course-approved JDK	Application
Build	Maven or Gradle	Build/dependencies
GUI	JavaFX	Desktop UI
Testing	JUnit 5	Automated tests
Logging	SLF4J + Logback	Diagnostics
Configuration	JSON/YAML/properties	Mappings and thresholds
Serialization	Jackson/Gson	Metadata/config
ML inference	ONNX Runtime Java or equivalent	Local inference
Input integration	OS-specific bridge/library	Touchpad events
Packaging	jpackage	Distribution

The exact ML runtime should be finalized after the training model is selected. The deployment path should remain local and Java-accessible.

16. Java Software Architecture

Package	Responsibility
com.glyvex.app	Application bootstrap and lifecycle
com.glyvex.input	Touchpad/input adapters
com.glyvex.gesture	Activation and state machine
com.glyvex.processing	Normalization and features
com.glyvex.ml	Model loading and inference
com.glyvex.commands	Command registry/execution
com.glyvex.config	Persistent settings
com.glyvex.ui	JavaFX screens and status
com.glyvex.logging	Diagnostics
com.glyvex.test	Tests and replay fixtures

> This separation prevents the common failure mode of putting hardware handling, ML, UI and process execution into one large Java class.

17. Command Mapping Architecture

The ML classifier should return a semantic command identifier rather than arbitrary executable text.

Stage	Example
Classifier output	OPEN_BROWSER
Registry lookup	OPEN_BROWSER → configured browser
Validation	Identifier is allowlisted
Execution	Java launches configured application
Feedback	UI reports success/failure

Possible benign demonstration mappings include calculator, text editor, browser, a predefined folder, or another locally installed application.

> **Important:** a misclassified gesture must never become an unrestricted shell command.

18. User Interface and UX

The permanent UI should be minimal because the touchpad is the primary interaction surface.

UI element	Purpose
Status indicator	Monitoring / Armed / Gesture Mode / Recognizing / Executing
Activation progress	Shows corner sequence progress
Gesture preview	Optional small trajectory display
Prediction	Recognized symbol and confidence
Mapping editor	Symbol-to-command configuration
Threshold settings	Tolerance, timeout and confidence threshold
Replay/test control	Test recorded trajectories

The interface should communicate system state without turning the project into an unnecessary dashboard application.

19. Hardware and Software Resources

Resource	Requirement / purpose
Laptop	Physical touchpad-equipped development machine
CPU	Modern multi-core processor
RAM	8 GB minimum; 16 GB recommended
Storage	SDK, IDE, datasets, models and builds
Touchpad	Must expose required input events
JDK	Java 17+ or course-approved version
IDE	IntelliJ IDEA / Eclipse / VS Code
Version control	Git + GitHub
ML tooling	Python/scikit-learn/PyTorch if permitted
Inference	ONNX Runtime Java or equivalent
Packaging	jpackage

Training and deployment environments should be distinguished. The final delivered application remains a Java program even if Python is used during model development.

20. Development Tools and Resources

Category	Tool / resource	Use
Java	OpenJDK	Compiler/runtime
IDE	IntelliJ IDEA / VS Code	Development
Build	Maven / Gradle	Build automation
UI	JavaFX	Desktop interface
ML experiments	scikit-learn / PyTorch	Training if permitted
Inference	ONNX Runtime Java	Local prediction
Testing	JUnit 5	Tests
Logging	SLF4J + Logback	Diagnostics
Repository	Git + GitHub	Collaboration
Documentation	Markdown + diagrams	Technical record
Packaging	jpackage	Application distribution

Record exact versions of major dependencies in the final report to improve reproducibility.

21. Development Timeline

Phase	Work	Deliverable
1	Requirements + input feasibility	Feasibility proof
2	Touchpad event acquisition	Raw input prototype
3	Four-corner state machine	Activation module
4	Capture + preprocessing	Normalized pipeline
5	Dataset preparation	Labeled data
6	Baseline ML	Initial classifier
7	Model evaluation	Selected model + metrics
8	Java inference	Local prediction module
9	Command registry	Safe command layer
10	UI/configuration	Desktop interface
11	Integration testing	End-to-end prototype
12	Packaging/documentation	Final demo + report

22. Testing Strategy

Test level	Target	Example
Unit	Individual functions	Coordinate normalization
State	State transitions	Wrong corner resets sequence
Input	Event interpretation	Touch event parsing
ML	Classifier	Known trajectory prediction
Integration	Full pipeline	Touch → model → command
Safety	Command validation	Unknown class rejected
Performance	Latency	End capture to prediction
User	Physical usability	Multiple users
Regression	Previous bugs	Trajectory replay

A trajectory replay harness should be included. It makes model and preprocessing tests reproducible without repeatedly drawing gestures on the physical touchpad.

23. Failure Modes and Mitigation

Failure mode	Likely cause	Mitigation
No touch events	OS/library limitation	Validate input bridge first
False activation	Accidental corner taps	Tolerance + sequence + timeout
Missed activation	Noise/timing	Configurable thresholds
Poor recognition	Insufficient data	More diverse samples
Symbol confusion	Similar shapes	Improve features/data
Slow inference	Heavy model	Smaller/optimized model
Wrong command	Low confidence	Confidence threshold
Command failure	Missing application	Validate configuration
Stuck state	Transition bug	Central FSM + recovery
Dataset leakage	Bad split	User-independent evaluation

24. Security and Privacy

- Keep the default command set small and allowlisted.
- Never execute raw ML output as a shell command.
- Reject unknown and low-confidence predictions.
- Keep raw touch trajectories local by default.
- Store only data required for debugging, training or evaluation.
- Provide an emergency disable mechanism.
- Validate imported configuration before execution.
- Use benign actions for the final demonstration.

Connecting an ML classifier to system commands creates a clear security boundary. The command layer must therefore be deliberately more restrictive than the classifier itself.

25. Repository Structure

Path	Contents
/src/main/java	Java application source
/src/main/resources	UI resources and configuration templates
/src/test/java	JUnit tests
/model	Exported ML model and metadata
/data/raw	Original dataset where permitted
/data/processed	Processed training/evaluation data
/scripts	Optional model/data preparation
/docs	Architecture and experiment records
/samples	Replayable trajectories
/config	Example command mappings
/README.md	Setup, build, run and demo instructions

Large datasets and model artifacts should not be blindly committed to Git. Use appropriate artifact storage if they become large.

26. Demonstration Scenario

Step	Demonstration
1	Launch GLYVEX and show Monitoring state.
2	Tap the four configured corners.
3	Show transition into Gesture Mode.
4	Draw a supported symbol.
5	Show captured trajectory and processing.
6	Display predicted class and confidence.
7	Resolve class through command registry.
8	Execute a benign desktop action.
9	Return to Monitoring.
10	Demonstrate rejection of an unsupported/low-confidence gesture.

Demonstrating a rejected gesture is important because it proves that uncertainty handling and safety logic are implemented.

27. Project Deliverables

Deliverable	Expected result
Java application	Runnable desktop application
ML model	Exported classifier + metadata
Dataset documentation	Source, labels, collection and split methodology
Source code	Modular Java implementation
Tests	Unit, state, ML and integration tests
Configuration	Gesture-to-command mappings
Architecture document	Components and data flow
Experiment report	Model comparison and evaluation
User guide	Installation and operation
Final presentation	Architecture + live demonstration
Final report	Problem, method, implementation, results and limitations

28. Future Extensions

- Personalized gesture profiles.
- Custom gesture-training interface.
- Multi-stroke symbols.
- Dynamic sequence models.
- Adaptive touchpad thresholds.
- Cross-platform input adapters.
- Application-specific command profiles.
- Model-error visualization.
- Optional voice confirmation for sensitive actions.
- Plugin architecture for additional command modules.

Future work should remain secondary to the core deliverable. A small, reliable gesture vocabulary with robust hardware input and defensible evaluation is preferable to an oversized feature list.

29. Team Responsibilities

Member	Suggested responsibility	Outputs
Ashwath Raj Mulakalapalli	System architecture + Java integration	Application structure, FSM, integration
BARATAM PRANEETH GUPTA	ML/data pipeline	Dataset, features, training, evaluation
Harsha Vardhan Kumar	Touch/input subsystem	Input bridge, corner detection, capture
Sai thanush reddy	UI, testing + documentation	JavaFX UI, configuration, testing, report

> This division is a suggested work allocation. All members should understand the complete pipeline for the final viva.

30. Technical Risks and Feasibility Gates

Gate	Question	Pass condition
G1	Can the target laptop expose touchpad events?	Coordinates/contact events captured
G2	Can four corners be distinguished?	Activation works consistently
G3	Can drawings be captured?	Representative trajectories recorded
G4	Can preprocessing stabilize them?	Comparable representations
G5	Can the model classify the classes?	Meaningful baseline
G6	Can Java run inference locally?	No remote prediction dependency
G7	Can commands be safely mapped?	Allowlisted execution
G8	Can errors recover?	System returns to safe state

Critical gate: G1. If the target system cannot expose the required touchpad data through standard Java APIs, use a native/helper input bridge rather than delaying the entire project.

31. Final Recommended Technology Stack

Component	Recommendation
Application	Java 17+ / course-approved JDK
UI	JavaFX
Build	Maven or Gradle
Input	OS-specific touchpad/native bridge behind Java interface
State management	Finite-state machine
Data representation	Timestamped normalized x/y trajectories
Preprocessing	Normalization + resampling + selected features
ML baseline	k-NN / Decision Tree / Random Forest
Candidate final model	SVM, Random Forest or compact neural model based on validation
Inference	ONNX Runtime Java or equivalent
Command layer	Allowlisted command registry
Testing	JUnit 5 + trajectory replay
Logging	SLF4J + Logback
Packaging	jpackage
Version control	Git + GitHub

> Engineering principle: keep the input pipeline deterministic, the model compact, the command layer restricted and the OS-specific code isolated.

32. Final Project Definition

GLYVEX is a Java-based touchpad interaction system that converts deliberate touch gestures into desktop commands. The user first activates the system through a four-corner touch sequence. Once activated, the user draws a predefined character or symbol on the touchpad.

The application records the trajectory, preprocesses it into a standardized representation, applies a machine-learning classifier, checks the prediction against a confidence threshold, and maps the recognized class to an explicitly configured action.

The project demonstrates the integration of **human–computer interaction, event-driven Java programming, finite-state machines, data preprocessing, machine learning, local inference, desktop application development and controlled system automation**.

> **Core pipeline:** Four-corner activation → Draw glyph → Capture trajectory → Preprocess → ML recognition → Confidence check → Command mapping → Safe execution
>
> **Team:** Ashwath Raj Mulakalapalli · BARATAM PRANEETH GUPTA · Harsha Vardhan Kumar · Sai thanush reddy
>
> **Primary constraints:** Java + ML dataset + physical touchpad.